数字图像处理

Digital Image Processing

主　讲：　赵歆波　计算机学院
邮　箱：　xbozhao@nwpu.edu.cn
QQ　：　66326191
QQ 群：　数字图像处理 654577569

第五章　图像增强

- 要点
  1. 直方图处理
  2. 平滑空间滤波器
  3. 锐化空间滤波器
  4. 低通频域滤波器
  5. 高通频域滤波器
  6. 思考与作业题

图像增强：空间运算

- 图像平滑
- 图像锐化
- 中值滤波

图像平滑—邻域平均

- 有些噪音是随机性的，它们对某一个像素的影响可以看作是孤立的。

  不具有统计特性

- 因此，和邻域各点相比，该点灰度值将有显著的不同。

  具有空间特性

- 因此，我们可以用邻域平均的方法来判断每一点是否含有噪音。并用适当的方法消除所发现的噪音。

图像平滑—噪声平滑

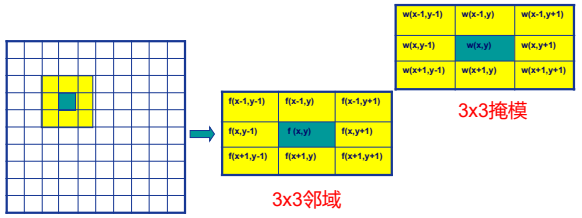
```
% 噪声生成函数imnoise简介
% 语法格式：J=imnoise(I,type,parameters);将类型为type参数为parameter的噪声添加到图像I中
% type取值：gaussian（高斯噪声）,localvar（均值为0的高斯噪声）,poisson（泊松噪声）,salt & pepper（椒盐噪声）,speckle（乘性噪声）
I=imread('lena.png');
% 显示原始图像
subplot(2,2,1),imshow(I),title('原始图像');
% 叠加均值为0，方差为0.02的高斯噪声
J1=imnoise(I,'gaussian',0,0.02);
subplot(2,2,2),imshow(J1),title('高斯噪声图像');
% 叠加密度为0.04的椒盐噪声
J2=imnoise(I,'salt & pepper',0.04);
subplot(2,2,3),imshow(J2),title('椒盐噪声图像');
% 叠加密度为0.04的乘法噪声
J3=imnoise(I,'speckle',0.04);
subplot(2,2,4),imshow(J3),title('乘法噪声图像');
```

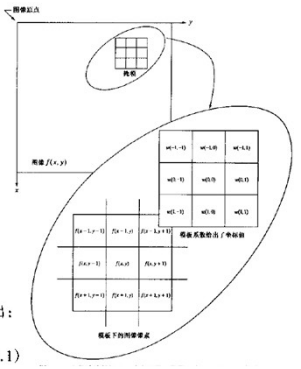


图像平滑—邻域平均

$$R = w(-1,-1)f(x-1,y-1) + w(-1,0)f(x-1,y) + \cdots + w(0,0)f(x,y) + \cdots + w(1,0)f(x+1,y) + w(1,1)f(x+1,y+1)$$

- 设当前的待处理像素为f(x,y)
- 子图像称为滤波器、掩模、核、模板或窗口





一般来说，在 $M \times N$ 的图像 f 上，用 $m \times n$ 大小的滤波器掩模进行线性滤波由下式给出：

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t) \qquad (3.5.1)$$

邻域平均的数学描述

- 定义
  - 把平均化处理看作一个作用于图象f(x,y)的低通空间滤波器，该滤波器的脉冲应为W(s,t)，于是滤波输出或新的数字图象可以用如下离散卷积表示

$$\hat{f}(x,y) = \sum_{s=-k}^{k} \sum_{t=-l}^{l} f(x-s, y-t) W(s,t)$$

- **k,l 据所选邻域大小来决定，一般来说,k=l=1（即3x3的小邻域）效果就很好了。邻域取得过大，会使灰度突变的一些有用信息，如景物边缘等，变得模糊起来。**
- **公式中W(s,t)为加权函数，称为掩模**

- 常用的掩模

$$H_1 = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

平均平滑（八邻域）

$$H_2 = \frac{1}{10}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

平均平滑+中心锐化

$$H_3 = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

平均平滑+中心锐化+四邻域锐化

$$H_4 = \frac{1}{8}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

平均平滑+中心抑制

$$H_5 = \frac{1}{2}\begin{bmatrix} 0 & \frac{1}{4} & 0 \\ \frac{1}{4} & 1 & \frac{1}{4} \\ 0 & \frac{1}{4} & 0 \end{bmatrix}$$

四邻域平均平滑+中心锐化

图像平滑—邻域平均

- 掩模的取法不同，中心点或邻域的重要程度也不相同。但所有的掩模必须保证全部权系数之和为单位值。
- 邻域平均的方法可以看作是一个作用于图像的低通空间滤波器。
  平滑滤波
- 掩模领域一般取3*3。如果取得过大，会使灰度突变的地方，变得模糊起来。



3x3掩模

空域增强-平滑滤波

```
% fspecial&filter2函数简介
% 语法格式：J=filter2(A,X,'shape');
% 使用矩阵A中的二维滤波器对数据X滤波，J的大小由shape（可选项）
决定。
% 语法格式：J=fspecial(type,parameters);
% type用于指定滤波器类型，parameters是与指定的滤波器相关的参数
。
I=imread('lena.png');
% 显示原始图像
subplot(231),imshow(I),title('原始图像');
% 叠加密度为0.02的椒盐噪声
I1=imnoise(I,'salt & pepper',0.02);
subplot(232),imshow(I1),title('椒盐噪声图像');
% 采用3x3模板平滑滤波
k1=filter2(fspecial('average',3),I1)/255;
% 采用5x5模板平滑滤波
k2=filter2(fspecial('average',5),I1)/255;
% 采用7x7模板平滑滤波
k3=filter2(fspecial('average',7),I1)/255;
% 采用9x9模板平滑滤波
k4=filter2(fspecial('average',9),I1)/255;
subplot(233),imshow(k1),title('3x3模板平滑滤波');
subplot(234),imshow(k2),title('5x5模板平滑滤波');
subplot(235),imshow(k3),title('7x7模板平滑滤波');
subplot(236),imshow(k4),title('9x9模板平滑滤波');
```





可以添加不同噪声，采用不同滤波模板，不同模板大小，观察程序运行的结果。

图像锐化—微分算子

图象锐化的目的是加强图象中景物的边缘和轮廓。锐化的作用是要使灰度反差增强。因为边缘和轮廓都位于灰度突变的地方。所以锐化算法的实现是基于微分作用。

一阶微分：梯度算子
二阶微分：拉普拉斯算子

强调灰度突变

图像锐化—梯度算子

❖定义

$$G[f(x,y)]=\begin{bmatrix}\dfrac{\partial f}{\partial x}\\ \dfrac{\partial f}{\partial y}\end{bmatrix}$$

- 梯度幅度

$$G[f(x,y)]=\sqrt{\left(\frac{\partial f}{\partial x}\right)^2+\left(\frac{\partial f}{\partial y}\right)^2}$$

- 向量的幅角

$$\theta_M=tg^{-1}\left[\frac{\partial f}{\partial y}/\frac{\partial f}{\partial x}\right]$$

❖将图像坐标系旋转

$$x=x'\cos\theta-y'\sin\theta$$
$$y=x'\sin\theta+y'\cos\theta$$

●对x',y'取偏导

$$\frac{\partial f}{\partial x'}=\frac{\partial f}{\partial x}\frac{\partial x}{\partial x'}+\frac{\partial f}{\partial y}\frac{\partial y}{\partial x'}=\frac{\partial f}{\partial x}\cos\theta+\frac{\partial f}{\partial y}\sin\theta$$

$$\frac{\partial f}{\partial y'}=\frac{\partial f}{\partial x}\frac{\partial x}{\partial y'}+\frac{\partial f}{\partial y}\frac{\partial y}{\partial y'}=-\frac{\partial f}{\partial x}\sin\theta+\frac{\partial f}{\partial y}\cos\theta$$

$$\left(\frac{\partial f}{\partial x'}\right)^2+\left(\frac{\partial f}{\partial y'}\right)^2=\left(\frac{\partial f}{\partial x}\right)^2+\left(\frac{\partial f}{\partial y}\right)^2$$

- 梯度幅度$G[f(x,y)]$是一个各向同性的微分算子

图像锐化—梯度算子

- 对数字图像，微分近似用差分代替

$$\Delta_x f(i,j)=f(i+1,j)-f(i,j)$$
$$\Delta_y f(i,j)=f(i,j+1)-f(i,j)$$

- 与x轴成任意夹角的差分（旋转差分）

$$\Delta_\theta f(i,j)=\Delta_x f(i,j)\cos\theta+\Delta_y f(i,j)\sin\theta$$

- 数字梯度向量的幅度

$$G[f(i,j)]=[\Delta_\theta f(i,j)]_{\max}=\sqrt{(\Delta_x f)^2+(\Delta_y f)^2}$$

$$\Delta_x f=\Delta_x f(i,j)$$
$$\Delta_y f=\Delta_y f(i,j)$$

- 向量的幅角

$$\theta_M=tg^{-1}\left[\Delta_y f/\Delta_x f\right]$$

- 数字梯度向量

$$\overline{G}[f(i,j)]=\begin{bmatrix}\Delta_x f(i,j)\\ \Delta_y f(i,j)\end{bmatrix},$$
$$i=0,1,\cdots,N-1$$
$$j=0,1,\cdots,N-1$$

图像锐化——一阶梯度算子

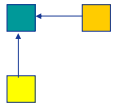
空天地海一体化大数据应用技术国家工程实验室
National Engineering Laboratory for Integrated Air-Space-Ground-Ocean Big Data Application Technology
陕西省语音与图像信息处理重点实验室

❖各向同性的一阶梯度算法

$$G[f(i,j)]=|f(i+1,j)-f(i,j)|+|f(i,j+1)-f(i,j)|$$

$$G[f(i,j)]=\max(|f(i+1,j)-f(i,j)|,|f(i,j+1)-f(i,j)|)$$

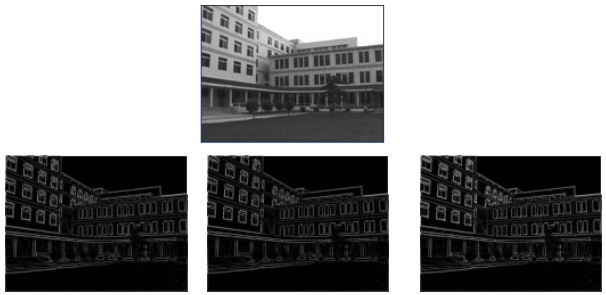
$$G[f(i,j)]=\{[f(i+1,j)-f(i,j)]^2+[f(i,j+1)-f(i,j)]^2\}^{1/2}$$



f(i-1,j-1)	f(i-1,y)	f(i-1,j+1)
f(i,j-1)	f (i,j)	f(i,j+1)
f(i+1,j-1)	f(i+1,j)	f(i+1,j+1)

各向同性一阶锐化效果图例

空天地海一体化大数据应用技术国家工程实验室
National Engineering Laboratory for Integrated Air-Space-Ground-Ocean Big Data Application Technology
陕西省语音与图像信息处理重点实验室



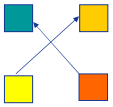
一阶梯度算法

空天地海一体化大数据应用技术国家工程实验室
National Engineering Laboratory for Integrated Air-Space-Ground-Ocean Big Data Application Technology
陕西省语音与图像信息处理重点实验室

❖交叉梯度算法（Roberts梯度算法）

$$G[f(i,j)]=|f(i+1,j+1)-f(i,j)|+|f(i+1,j)-f(i,j+1)|$$

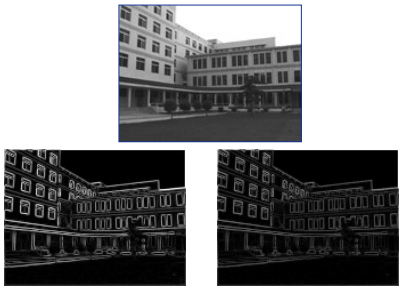
$$G[f(i,j)]=\max(|f(i+1,j+1)-f(i,j)|,|f(i+1,j)-f(i,j+1)|)$$



f(i-1,j-1)	f(i-1,y)	f(i-1,j+1)
f(i,j-1)	f (i,j)	f(i,j+1)
f(i+1,j-1)	f(i+1,j)	f(i+1,j+1)

Roberts梯度锐化效果图例

空天地海一体化大数据应用技术国家工程实验室
National Engineering Laboratory for Integrated Air-Space-Ground-Ocean Big Data Application Technology
陕西省语音与图像信息处理重点实验室



图像锐化—梯度图像生成

- 方法一：

$$g(i,j)=G[f(i,j)]$$

- 方法二：

$$g(i,j)=\begin{cases}G[f(i,j)] & \text{当}G[f(i,j)]\geq T\text{时}\\ f(i,j) & \text{其它}\end{cases}$$

- 方法三：

$$g(i,j)=\begin{cases}L_G\ \text{白} & \text{当}G[f(i,j)]\geq T\text{时}\\ f(i,j) & \text{其它}\end{cases}$$

- 方法四：

$$g(i,j)=\begin{cases}G[f(i,j)] & \text{当}G[f(i,j)]\geq T\text{时}\\ L_B\ \text{黑} & \text{其它}\end{cases}$$

- 方法五：

$$g(i,j)=\begin{cases}L_G\ \text{白} & \text{当}G[f(i,j)]\geq T\text{时}\\ L_B\ \text{黑} & \text{其它}\end{cases}$$

空间运算---梯度锐化

阈值不同





T=10



T=40

空域增强-梯度锐化

方法不同

```
clear all
I=imread('lena.png');
subplot(231),imshow(I),title('原始图像');
% 图像类型转换
I=double(I);

% 计算梯度Gx,Gy
[Gx Gy]=gradient(I);
% 求梯度幅度，是矩阵点乘
G=sqrt(Gx.*Gx+Gy.*Gy);

% 方法一，直接显示梯度
J1=G;
subplot(232),imshow(J1,[]),title('梯度增强显示方法一');

% 方法二，阈值T=20
% 适当地选取阈值，明显的边缘轮廓得到突出，又不会破坏原来的灰度变化比较平缓的背景。
T=20;
J2=I;
K=find(G>=T);
J2(K)=G(K);
subplot(233),imshow(J2,[]),title('梯度增强显示方法二');
```

```
% 方法三，阈值T=20，LG=255
% LG是根据需要制定一个灰度级，将明显边缘用一个固定的灰度级LG表示。
LG=255;
J3=I;
K=find(G>=T);
J3(K)=LG;
subplot(234),imshow(J3,[]),title('梯度增强显示方法三');

% 方法四，阈值T=20，LB=0
% 该方法将背景用一个固定的灰度级LB来表示，以便于研究边缘灰度的变化。
% 相当于LB=0，J4=LB*I
J4=zeros(size(I));
K=find(G>=T);
J4(K)=G(K);
subplot(235),imshow(J4,[]),title('梯度增强显示方法四');

% 方法五，阈值T=20，LB=0，LG=255
% 该方法将背景和明显边缘分别用灰度级LB和LG表示，生成二值图像，便于研究边缘所在的位置。
J5=zeros(size(I));
K=find(G>=T);
J5(K)=LG;
subplot(236),imshow(J5,[]),title('梯度增强显示方法五');
```

空域增强-梯度锐化













针对不同的图像应用可选取不同的参数。

基于拉普拉斯算子的锐化处理

❖定义

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

● 差分形式

$$g(i,j) = 4f(i,j) - f(i+1,j) - f(i-1,j) - f(i,j+1) - f(i,j-1)$$

●卷积形式

$$g(i,j) = \sum_{r=-k}^{k}\sum_{s=-l}^{l} f(i-r, j-s)H(r,s)$$

➢ 常用的掩模

$$H_1 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

四邻域算子

$$H_2 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

八邻域算子

$$H_3 = \begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}$$

八邻域算子

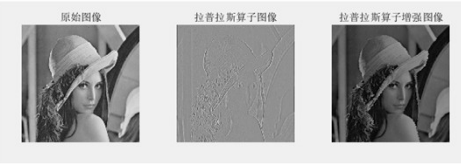
$$H_4 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

四邻域算子

空域增强-拉普拉斯算子增强

```
clear all
I=imread('lena.bmp');
subplot(131),imshow(I),title('原始图像');

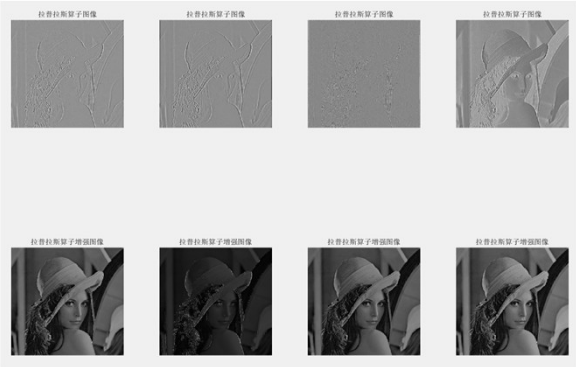
% 类型转换，因为conv2函数的参数要求为double型
I=double(I);
H=[0 1 0,1 -4 1,0 1 0];
J=conv2(I,H,'same');
K=find(J>=150)
J2=I;
J2(K)=J(K);
subplot(132),imshow(J,[]),title('拉普拉斯算子图像');
subplot(133),imshow(J2,[]),title('拉普拉斯算子增强图像');
```



空域增强-拉普拉斯算子增强

```
clear all
I=imread('lena.bmp');
%subplot(331),imshow(I),title('原始图像');
% 类型转换，因为conv2函数的参数要求为double型
I=double(I);
H1=[0 1 0,1 -4 1,0 1 0];
H2=[1 1 1,1 -8 1,1 1 1];
H3=[-1 2 -1,2 -4 2,-1 2 -1];
H4=[0 1 0,1 -5 1,0 1 0];
J1=conv2(I,H1,'same');
J2=conv2(I,H2,'same');
J3=conv2(I,H3,'same');
J4=conv2(I,H4,'same');
K1=find(J1>=150)
K2=find(J2>=150)
K3=find(J3>=150)
K4=find(J4>=150)
I1=I;
I2=I;
I3=I;
I4=I;
I1(K1)=J1(K1);
I2(K2)=J2(K2);
I3(K3)=J3(K3);
I4(K4)=J4(K4);
subplot(241),imshow(J1,[]),title('拉普拉斯算子图像');
subplot(242),imshow(J2,[]),title('拉普拉斯算子图像');
subplot(243),imshow(J3,[]),title('拉普拉斯算子图像');
subplot(244),imshow(J4,[]),title('拉普拉斯算子图像');
subplot(245),imshow(I1,[]),title('拉普拉斯算子增强图像');
subplot(246),imshow(I2,[]),title('拉普拉斯算子增强图像');
subplot(247),imshow(I3,[]),title('拉普拉斯算子增强图像');
subplot(248),imshow(I4,[]),title('拉普拉斯算子增强图像');
```

算子不同



Sobel算子

在边缘检测中，常用的一种模板是Sobel算子。

Sobel 算子有两个，一个是检测水平边缘的； 另一个是检测垂直边缘的 。

$$S = \left(d_x^2 + d_y^2\right)^{\frac{1}{2}}$$

$$d_y = \left[f_{i-1,j-1} + 2f_{i,j-1} + f_{i+1,j-1}\right] - \left[f_{i-1,j+1} + 2f_{i,j+1} + f_{i+1,j+1}\right]$$ 垂直

$$d_x = \left[f_{i+1,j-1} + 2f_{i+1,j} + f_{i+1,j+1}\right] - \left[f_{i-1,j-1} + 2f_{i-1,j} + f_{i-11,j+1}\right]$$ 水平

$$d_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} \quad d_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Sobel算子效果图



Prewitt算子

在边缘检测中，常用的一种模板是Prewitt 算子。

Prewitt 算子有两个，一个是检测水平边缘的； 另一个是检测垂直边缘的 。

$$S_p = \left(d_x^2 + d_y^2\right)^{\frac{1}{2}}$$

$$d_y = \left[f_{i-1,j-1} + f_{i,j-1} + f_{i+1,j-1}\right] - \left[f_{i-1,j+1} + f_{i,j+1} + f_{i+1,j+1}\right]$$
$$d_x = \left[f_{i+1,j-1} + f_{i+1,j} + f_{i+1,j+1}\right] - \left[f_{i-1,j-1} + f_{i-1,j} + f_{i-11,j+1}\right]$$

$$d_x = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \qquad d_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Prewitt算子效果图



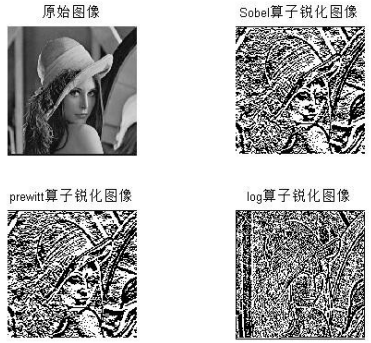
空域增强-其他算子

```
clear all
I=imread('lena.png');
subplot(221),imshow(I),title('原始图像');

H=fspecial('Sobel');
I2=filter2(H,I);
subplot(222),imshow(I2),title('Sobel算子锐化图像');

H1=fspecial('prewitt');
I3=filter2(H1,I);
subplot(223),imshow(I3),title('prewitt算子锐化图像');

H2=fspecial('log');
I4=filter2(H2,I);
subplot(224),imshow(I4),title('log算子锐化图像');
```



Isotropic算子

在边缘检测中，常用的一种模板是Isotropic算子。

Isotropic算子有两个，一个是检测水平边缘的；另一个是检测垂直边缘的。

$$I_s = \left(d_x^2 + d_y^2\right)^{\frac{1}{2}}$$

$$d_y = \left[f_{i-1,j-1} + \sqrt{2}f_{i,j-1} + f_{i+1,j-1}\right] - \left[f_{i-1,j+1} + \sqrt{2}f_{i,j+1} + f_{i+1,j+1}\right]$$
$$d_x = \left[f_{i+1,j-1} + \sqrt{2}f_{i+1,j} + f_{i+1,j+1}\right] - \left[f_{i-1,j-1} + \sqrt{2}f_{i-1,j} + f_{i-11,j+1}\right]$$

$$d_x = \begin{bmatrix} 1 & 0 & -1 \\ \sqrt{2} & 0 & -\sqrt{2} \\ 1 & 0 & -1 \end{bmatrix} \qquad d_y = \begin{bmatrix} -1 & -\sqrt{2} & -1 \\ 0 & 0 & 0 \\ 1 & \sqrt{2} & 1 \end{bmatrix}$$

Isotropic算子效果图



Kirsch算子

在边缘检测中，常用的一种模板是Kirsch算子。

$$g(i,j) = \max\{1, \max_{k=0}^{7}\{[|5S_k - 3T_k|]\}\}$$

$$S_k = A_k + A_{k+1} + A_{k+2}$$

$$T_k = A_{k+3} + A_{k+4} + A_{k+5} + A_{k+6} + A_{k+7}$$

A_0	A_1	A_2
A_7	F(i,j)	A_3
A_6	A_5	A_4

Kirsch算子 利用8个卷积模板计算出了某点8个方向的梯度幅值和方向，并以最大的卷积值作为该点的灰度值。

$$\begin{pmatrix} 5 & 5 & 5 \\ -3 & 0 & -3 \\ -3 & -3 & -3 \end{pmatrix}, \begin{pmatrix} -3 & 5 & 5 \\ -3 & 0 & 5 \\ -3 & -3 & -3 \end{pmatrix}, \begin{pmatrix} -3 & -3 & 5 \\ -3 & 0 & 5 \\ -3 & -3 & 5 \end{pmatrix}, \begin{pmatrix} -3 & -3 & -3 \\ -3 & 0 & 5 \\ -3 & 5 & 5 \end{pmatrix}$$
$$\begin{pmatrix} -3 & -3 & -3 \\ -3 & 0 & -3 \\ 5 & 5 & 5 \end{pmatrix}, \begin{pmatrix} -3 & -3 & -3 \\ 5 & 0 & -3 \\ 5 & 5 & -3 \end{pmatrix}, \begin{pmatrix} 5 & -3 & -3 \\ 5 & 0 & -3 \\ 5 & -3 & -3 \end{pmatrix}, \begin{pmatrix} 5 & 5 & -3 \\ 5 & 0 & -3 \\ -3 & -3 & -3 \end{pmatrix}$$

Kirsch 算子效果图



Wallis算子

$$g(i,j)=\log[f(i,j)]-\tfrac{1}{4}s$$

$$s=[\log f(i-1,j)+\log f(i+1,j)+\log f(i,j-1)+\log f(i,j+1)$$

该模板是将Laplacian微分算子和对数算子结合的一种锐化算子。因为人眼对画面信号的处理过程中有一个近似的对数运算环节，解决了Laplacian微分算子对画面比较暗的部分锐化比较弱的缺陷。

- 它可以看作是校正了视觉的指数特性后的拉普拉斯运算。

$$H_1=\begin{bmatrix}0&-1&0\\-1&4&-1\\0&-1&0\end{bmatrix} \Rightarrow H_1=\begin{bmatrix}0&-\frac{1}{4}&0\\-\frac{1}{4}&1&-\frac{1}{4}\\0&-\frac{1}{4}&0\end{bmatrix}$$

Wallis算法效果图



中值滤波（非线性）

- 抑制噪声：低通滤波
- 提升边缘轮廓：高通滤波

- **中值滤波**
  - **用一个窗口W在图像上扫描，把窗口内包含的图像像素按灰度升（或降）序排列起来，取灰度值居中的像素灰度为窗口中心像素的灰度**

$$g(m,n)=Median\{f(m-k,n-l),(k,l)\in\omega\}$$

中值滤波

- 给出滤波用的模板，如下图所示是一个一维的模板，待处理像素的灰度取这个模板中的灰度的中值。



例：

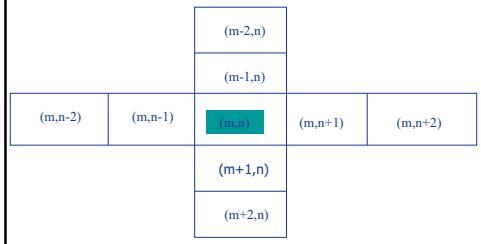
原图象为： 2 2 6 2 1 2 4 4 4 2 4

处理后为： 2 2 2 2 2 2 4 4 4 4 4

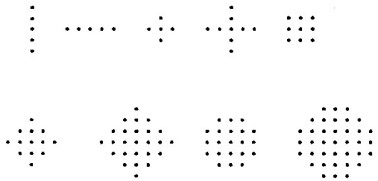
(1,2,2,2,6) (1,2,2,2,6) (1,2,2,4,6) (2,4,4)

中值滤波

下图是一个二维中值滤波器的模板示意图



❖中值滤波常用窗形状



中值滤波的效果



空间运算---中值滤波



3x1　　1x3　　3x3

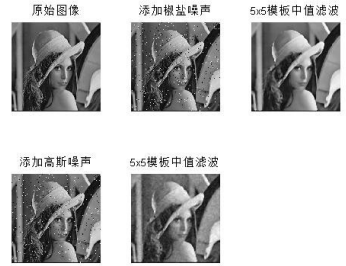
空域增强-中值滤波

❖线性组合中值滤波

$$g(m,n)=\sum_{K=1}^{M}a_K \underset{W_K}{Med}\left[f(i,j)\right]$$

$$g(m,n)=\underset{K}{Max}\left\{\underset{W_K}{Med}\left[f(i,j)\right]\right\}$$

```
% 中值滤波器函数medfilt2简介
% 语法格式：J=medfilt2(I,[m,n]);
% I为待处理图片，[m,n]为滤波窗口大小，默认为3x3
% 相对高斯噪声和乘性噪声，中值滤波器对于椒盐噪声的滤除效果最好。
I=imread('lena.png');
% 显示原始图像
subplot(231),imshow(I),title('原始图像');
% 给原始图像添加椒盐噪声
J=imnoise(I,'salt & pepper',0.03);
subplot(232),imshow(J),title('添加椒盐噪声');
% 椒盐噪声用5x5模板中值滤波
K=medfilt2(J,[5,5]);
subplot(233),imshow(K,[]),title('5x5模板中值滤波');
% 给原始图像添加高斯噪声
J2=imnoise(I,'gaussian',0.03);
subplot(234),imshow(J),title('添加高斯噪声');
% 高斯噪声用5x5模板中值滤波
K2=medfilt2(J2,[5,5]);
subplot(235),imshow(K2,[]),title('5x5模板中值滤波');
```

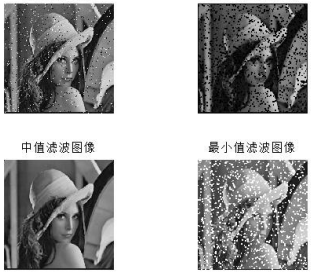


可以尝试添加不同的噪声，采用不同模板大小观察中值滤波结果。

空域增强-统计滤波

统计滤波器是测量图像的局部统计特征，然后用其统计值作为输出图像的像元值。其主要是通过计算小的邻近区(窗口范围内)的统计值，如其均值、中值、最大值、最小值等的改变来消除噪声。

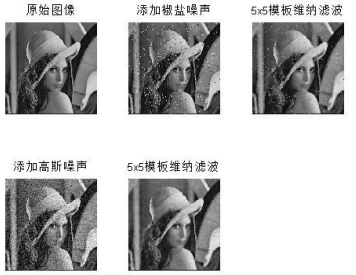
```
% 图像统计滤波函数ordfilt2简介
% 语法格式：B=ordfilt2(A,order,domain);
% 通过邻域domain中的第order个非零元素置换A中的每个元素。
% 运行结果会发现，当输出邻域最小值时，输出图像中存在很多暗色的噪声点；
% 当输出邻域最大值时，输出图像中存在很多亮色的噪声点。
clear all;
I=imread('lena.png');
N1=imnoise(I,'salt & pepper',0.04);
% 用3x3窗口实现最小值、中值、最大值滤波
G1=ordfilt2(N1,1,ones(3));
G2=ordfilt2(N1,5,ones(3));
G3=ordfilt2(N1,9,ones(3));
% 分别显示噪声图像和滤波图像
subplot(221),imshow(N1),title('椒盐噪声图像');
subplot(222),imshow(G1),title('最小值滤波图像');
subplot(223),imshow(G2),title('中值滤波图像');
subplot(224),imshow(G3),title( '最大值滤波图像');
```



空域增强-维纳滤波

维纳滤波器是用来自适应地（非线性）对图像中的噪声进行滤除，根据图像中的局部方差调节滤波器的输出，较线性滤波结果要好一些，能够更好地保存图像的边缘和高频细节信息。

```
% 维纳滤波器模板函数wiener2简介
% 语法格式：J=wiener2(I,[m n],noise);
% [m n]为滤波器窗口大小，默认值为3x3，noise用来指定噪声功率（可选项）
% 该函数用来自适应地对图像中的噪声进行滤除，根据图像中的局部方差调节滤波器
的输出，较线性滤波结果要好一些，能够更好地保存图像的边缘和高频细节信息。
I=imread('lena.png');
% 显示原始图像
subplot(231),imshow(I),title('原始图像');
% 添加均值为0，方差为0.03的椒盐噪声
J=imnoise(I,'salt & pepper',0.03);
subplot(232),imshow(J),title('添加椒盐噪声');
% 椒盐噪声&5x5模板维纳滤波
K=wiener2(J,[5 5]);
subplot(233),imshow(K,[]),title('5x5模板维纳滤波');
% 添加均值为0，方差为0.03的高斯噪声
J2=imnoise(I,'gaussian',0.03);
subplot(234),imshow(J2),title('添加高斯噪声');
% 高斯噪声&5x5模板维纳滤波
K2=wiener2(J2,[5 5]);
subplot(235),imshow(K2,[]),title('5x5模板维纳滤波');
```



可以尝试不同噪声、不同模板大小的维纳滤波。

空域增强-放大细化

❖ 重复细化

$$[f(i,j)]=\begin{bmatrix}1&2&3\\4&5&6\end{bmatrix} \rightarrow [g(i,j)]=\begin{bmatrix}1&1&2&2&3&3\\1&1&2&2&3&3\\4&4&5&5&6&6\\4&4&5&5&6&6\end{bmatrix}$$

$$H=\begin{bmatrix}1&1\\1&1\end{bmatrix}$$

$$[f(i,j)]=\begin{bmatrix}1&2&3\\4&5&6\end{bmatrix} \rightarrow \boxed{\text{插零}} \rightarrow \begin{bmatrix}1&0&2&0&3&0\\0&0&0&0&0&0\\4&0&5&0&6&0\\0&0&0&0&0&0\end{bmatrix} \rightarrow \boxed{\text{和}H\text{卷积}} \rightarrow \begin{bmatrix}1&1&2&2&3&3\\1&1&2&2&3&3\\4&4&5&5&6&6\\4&4&5&5&6&6\end{bmatrix}$$

THANK YOU!